

Midterm 1 Review

CSCI 1101

2024-02-16

About the Exam

Specifics:

- In class next Tuesday.
- Paper-and-pencil based.
- You may bring one index card (or 1/2 sheet of notebook paper) with notes.
- Covers material up to and including lists without mutation (Tuesday's lecture).

Resources:

- lecture notes, homeworks, in-class activities
- practice exam on canvas
- CodeRunner Exam Practice (up to Mixed Practice 1)

Names

What are python's rules for naming variables and functions?

Names are composed of only:

- letters (any case)
- numerals
- underscores (_)

Can't start with a numeral and can't be a keyword.

What makes for a good name?

Should be descriptive, words separated by underscores.

Assignment Statements

Which of these are valid *assignment statements*?

If valid, what do they mean? If not, why not?

- $x = 6 * 7$
- $6 * 7 = x$
- $x == 6 * 7$
- $xs = [1, 2, 3]$

Operators and Types

Strictly speaking, the operators `_+_, _-_, *_1_, _//_, _%_, _/__, _<_`, etc. should be used with arguments of the *same* type.

However, in some cases python is forgiving and lets you use them with a mix of numeric types. In this case the `ints` are implicitly converted to `floats`.

```
2 + 3.5      # float(2) + 3.5
1.5 < 3      # 1.5 < float(3)
```

Similarly, when testing equality with `_==_` or `_!=_`, python will implicitly convert the `ints` to `floats`, but you may assume all other mixed-type equalities are `False`.

```
2 == 2.0      # float(2) == 2.0
"True" != True # no implicit conversion
"10" != 10    # no implicit conversion
```

¹when used for multiplication

Returning vs Printing

Consider the following functions:

```
def return_it(arg) :  
    return arg
```

```
def print_it(arg) :  
    print(str(arg))
```

What happens when we do the following:

```
x = return_it('Bowdoin')  
y = print_it(42)
```

What are the values of x and y?

Booleans

What is wrong with the following function?

```
def stock_check(item) :  
    if item == "iphone" or "puppy" :  
        return "sold-out"  
    else :  
        return "in-stock"
```

What are two ways to fix it?

Conditionals

Consider the following functions:

```
def describe_1(num) :  
    if num % 2 == 0 :  
        print("even")  
    elif num > 0 :  
        print("positive")  
    else :  
        print("boring")
```

```
def describe_2(num) :  
    if num % 2 == 0 :  
        print("even")  
    if num > 0 :  
        print("positive")  
    else :  
        print("boring")
```

What are some arguments for which these functions behave differently? Why?

Strings

Write a function `initials : (str , str , str) --> str` that takes three nonempty strings, presumed to be a person's first, middle, and last names, and returns the string of their initials.

For example:

```
> initials("Franklin" , "Delano" , "Roosevelt")  
'F. D. R.'  
> initials("Martin" , "Luther" , "King")  
'M. L. K.'
```

Hint: use string interpolation.

More Strings

Write a function `middle_part` : `(str) --> str`
that takes a string argument and returns the substring that excludes the first and last characters (if any).

For example:

```
> middle_part("Bowdoin")  
'owdoi'  
> middle_part("Hi")  
''
```

Lists

Write a function

`contains_at_least : (list a , int , a) --> bool`

that returns `True` when the specified list contains at least the specified number of copies of the specified element.

For example:

```
> contains_at_least(list('bowdoin') , 2 , 'o')
```

```
True
```

```
> contains_at_least([True, False] , 2 , True)
```

```
False
```

More Lists

Write a function

`biggest_in_longest` : `(list int , list int) --> int`
that returns the largest element in the longer of the argument lists.
(If the list lengths are equal then the first list is chosen by default.)

For example:

```
> biggest_in_longest([1, 2, 4, 3] , [2, 10])  
4  
> biggest_in_longest([], [2, 10])  
10
```

Hint 1: the function `max` : `(list a) --> a` returns the largest list element.

Hint 2: try using a *conditional expression*.

To Do

Study for Midterm 1 in class next Tuesday, February 20.

Finish *Homework 4: Strings and Lists* on CodeRunner by 11:00AM next Thursday.

Finish on *Project 1: The Turtle Module* on CodeRunner by next Friday.